Lab #3: xv6 Threads

Team Members

- **Manoj Manjunatha**: 862481448
- **Vijay Ram Enaganti**: 862545736
- **Subhash Bangalore Sateesha**: 862541575

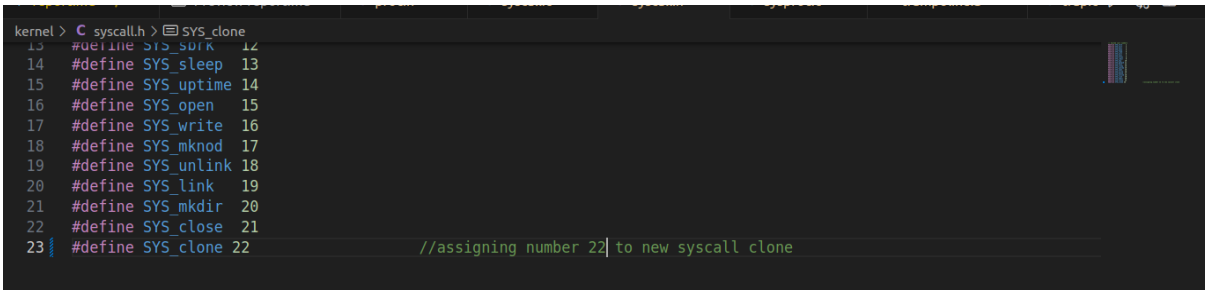
Demo Video: https://youtu.be/uL2p1AMMSb4

List of All Files Modified:

- kernel
  - syscall.h
  - syscall.c
  - sysproc.c
  - proc.h
  - proc.c
  - defs.h
  - trap.c
- user
  - usys.pl
  - thread.c
  - thread.h
  - user.h
- MAKEFILE

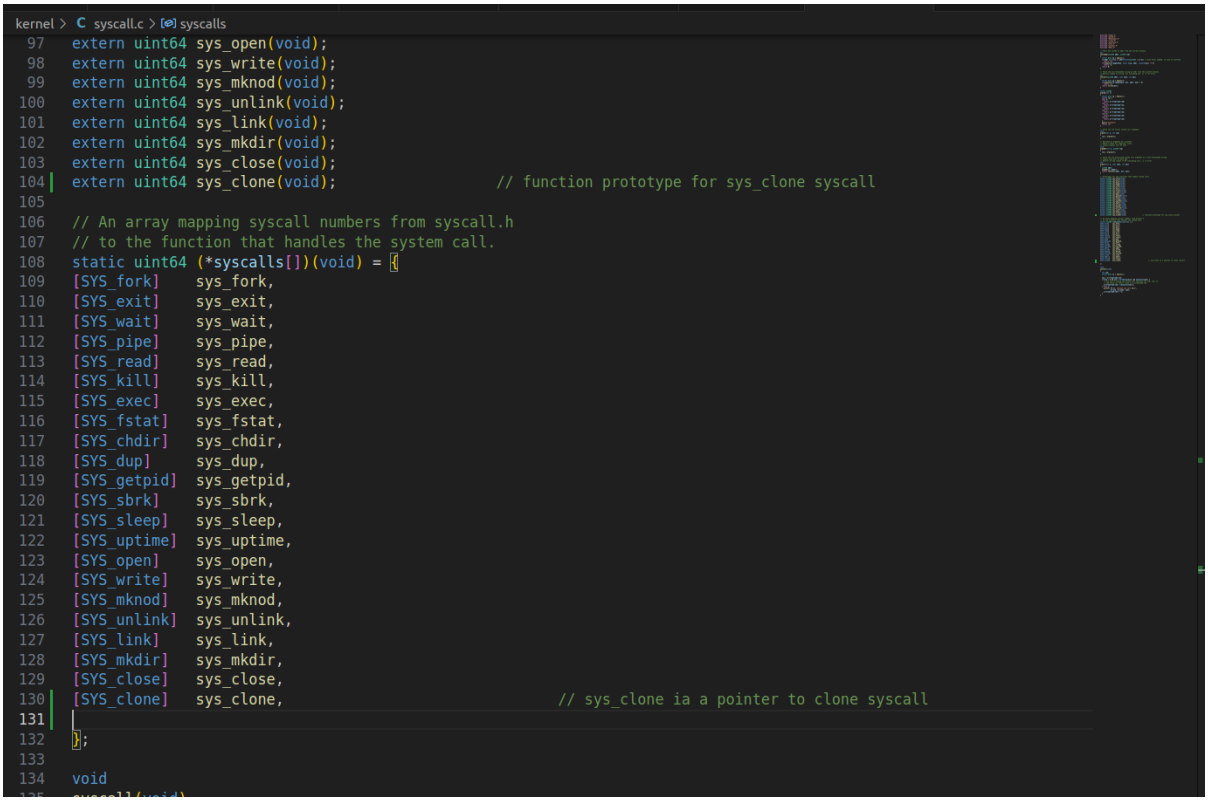
Changes Explained

- syscall.h
  - In the `syscall.h` file, the number **22** is assigned to the `sys_clone` syscall. This number serves as an index into the array of function pointers defined in `syscall.c`.



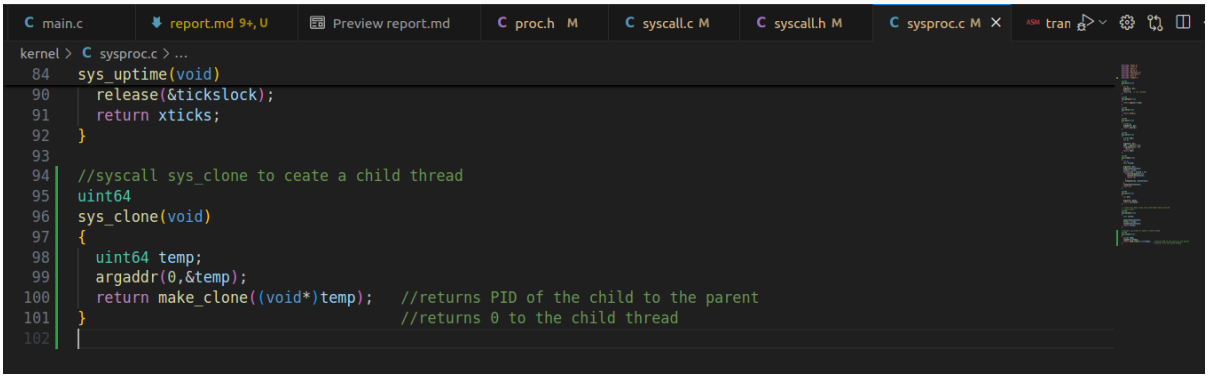
- syscall.c
  - In syscall.c, there is an array of function pointers, where each index corresponds to a specific system call. To support the clone system call, we added a pointer to the sys_clone function at the appropriate index in this array. Additionally, we included the function prototype for

sys_clone in syscall.c. The changes are as follows:



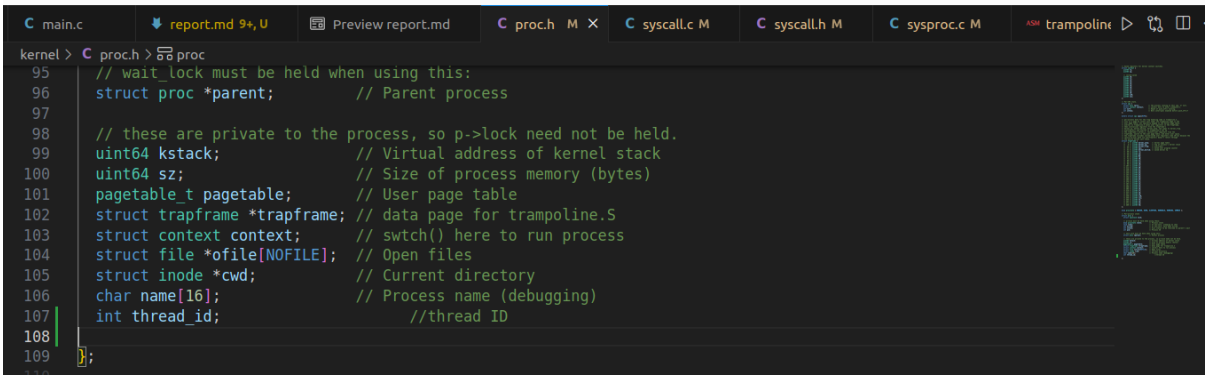
```
kernel > C syscall.c > [@] syscalls
 97  extern uint64 sys_open(void);
 98  extern uint64 sys_write(void);
 99  extern uint64 sys_mknod(void);
100  extern uint64 sys_unlink(void);
101  extern uint64 sys_link(void);
102  extern uint64 sys_mkdir(void);
103  extern uint64 sys_close(void);
104  extern uint64 sys_clone(void);                // function prototype for sys_clone syscall
105
106  // An array mapping syscall numbers from syscall.h
107  // to the function that handles the system call.
108  static uint64 (*syscalls[])(void) = {
109  [SYS_fork]    sys_fork,
110  [SYS_exit]    sys_exit,
111  [SYS_wait]    sys_wait,
112  [SYS_pipe]    sys_pipe,
113  [SYS_read]    sys_read,
114  [SYS_kill]    sys_kill,
115  [SYS_exec]    sys_exec,
116  [SYS_fstat]   sys_fstat,
117  [SYS_chdir]   sys_chdir,
118  [SYS_dup]     sys_dup,
119  [SYS_getpid]  sys_getpid,
120  [SYS_sbrk]    sys_sbrk,
121  [SYS_sleep]   sys_sleep,
122  [SYS_uptime]  sys_uptime,
123  [SYS_open]    sys_open,
124  [SYS_write]   sys_write,
125  [SYS_mknod]   sys_mknod,
126  [SYS_unlink]  sys_unlink,
127  [SYS_link]    sys_link,
128  [SYS_mkdir]   sys_mkdir,
129  [SYS_close]   sys_close,
130  [SYS_clone]   sys_clone,                          // sys_clone ia a pointer to clone syscall
131
132  };
133
134  void
```

- sysproc.c
  - In sysproc.c, we implement the system call functions. We have defined the sys_clone function, which returns the child's PID to the parent and 0 to the child thread. Within sys_clone, the argument passed represents the address of the child thread. We use the built-in argaddr function to retrieve this address from the user space and pass it into the kernel function.



```
C main.c   report.md 9+, U   Preview report.md   C proc.h M   C syscall.c M   C syscall.h M   C sysproc.c M ×   tran
kernel > C sysproc.c > ...
 84  sys_uptime(void)
 90    release(&tickslock);
 91    return xticks;
 92  }
 93
 94  //syscall sys_clone to ceate a child thread
 95  uint64
 96  sys_clone(void)
 97  {
 98    uint64 temp;
 99    argaddr(0,&temp);
100    return make_clone((void*)temp);   //returns PID of the child to the parent
101  }                                   //returns 0 to the child thread
102
```

- proc.h
  - In proc.h, we have added a new variable thread_id to the Process Control Block (PCB) structure to support thread management.



```
C main.c   report.md 9+, U   Preview report.md   C proc.h M ×   C syscall.c M   C syscall.h M   C sysproc.c M   trampolin
kernel > C proc.h > proc
 95  // wait_lock must be held when using this:
 96  struct proc *parent;          // Parent process
 97
 98  // these are private to the process, so p->lock need not be held.
 99  uint64 kstack;                // Virtual address of kernel stack
100  uint64 sz;                    // Size of process memory (bytes)
101  pagetable_t pagetable;        // User page table
102  struct trapframe *trapframe; // data page for trampoline.S
103  struct context context;       // swtch() here to run process
104  struct file *ofile[NOFILE];  // Open files
105  struct inode *cwd;            // Current directory
106  char name[16];                // Process name (debugging)
107  int thread_id;                      //thread ID
108
109  };
```

- proc.c

- In this file, we implement the main function make_clone(), which is responsible for creating threads. We initialize the nexttid variable to 1, which will be used to assign unique thread_ids to each thread. Additionally,we declare a struct spinlock named tid_lock to synchronize access to the thread ID assignment process.

```
16
17  int nextpid = 1;
18  int nexttid = 1;           //nexttid for thread
19
20  struct spinlock pid_lock;
21  struct spinlock tid_lock;   //struct for threads
22
```

- We are initializing nexttid in proc table.

```
54  void
55  procinit(void)
56  {
57    struct proc *p;
58
59    initlock(&pid_lock, "nextpid");
60    initlock(&wait_lock, "wait_lock");
61
62    initlock(&tid_lock, "nexttid");         //tid init
63
64    for(p = proc; p < &proc[NPROC]; p++) {
65        initlock(&p->lock, "proc");
66        p->state = UNUSED;
67        p->kstack = KSTACK((int) (p - proc));
68    }
69  }
70
```

- The alloctid() function is responsible for creating a trapframe for each thread. Within this function, we acquire the tid_lock before assigning a thread_id to ensure thread-safe access. After assigning the current value of nexttid as the thread_id, we increment nexttid for future threads and then release the lock. The function returns the assigned thread_id.

```
14
15  //function to create a trapframe for each thread
16  int
17  alloctid(){
18    int thread_id;
19    acquire(&tid_lock);     //acquire the lock
20    thread_id = nexttid;          //assign thread_id as nexttid
21    nexttid = nexttid + 1;  //incremnet nexttid
22    release(&tid_lock);     //release the lock
23    return thread_id;             //return thread_id to parent
24  }
25
```

- In allocproc() we initialize thread_id to 0 for each process.

```
130 static struct proc*
131 allocproc(void)
132 {
133   struct proc *p;
134
135   for(p = proc; p < &proc[NPROC]; p++) {
136     acquire(&p->lock);
137     if(p->state == UNUSED) {
138       goto found;
139     } else {
140       release(&p->lock);
141     }
142   }
143   return 0;
144
145 found:
146   p->pid = allocpid();
147   p->state = USED;
148
149   p->thread_id = 0;          //initialize thread_id as 0
150
```

- We implement a new function to allocate the necessary state for a thread to run in the kernel and return with the process lock held. If there is an unused entry in the process table, we allocate a thread_id for the new thread using the alloctid() function. Next, we allocate a

TRAPFRAME page for the thread and set up a new execution context that begins at forkret.

```
static struct proc*
allocproc_thread(void)
{
  struct proc *p;

  for(p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);
    if(p->state == UNUSED) {
      goto found;
    } else {
      release(&p->lock);
    }
  }
  return 0;

  found:
  p->pid = allocpid();
  p->state = USED;
  p->thread_id = alloctid();    //allocate thread_id of a thread
  // Allocate a trapframe page.
  if((p->trapframe = (struct trapframe *)kalloc()) == 0)
  {
    freeproc(p);
    release(&p->lock);
    return 0;
  }

  // Set up new context to start executing at forkret,
  // which returns to user space.
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)forkret;
  p->context.sp = p->kstack + PGSIZE;

  return p;
}
```

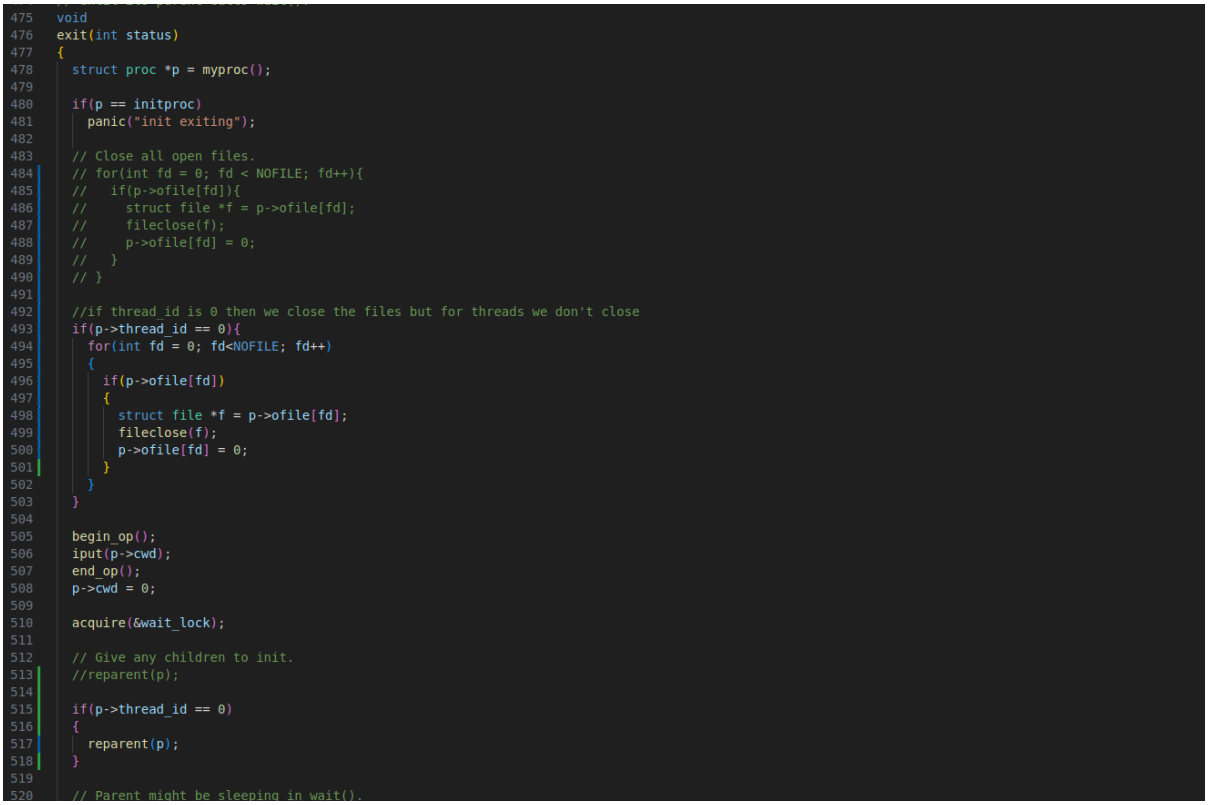
- In the freeproc() function, we deallocate child resources only if the thread_id is not 0 indicating that the process is a thread. In such cases, we use uvmunmap() to free the child thread's resources. If thread_id is 0, meaning it is a regular process, the default resource deallocation logic is executed.

```
// free a proc structure and the data hanging from it,
// including user pages.
// p->lock must be held.
static void
freeproc(struct proc *p)
{
  if(p->trapframe)
    kfree((void*)p->trapframe);
  p->trapframe = 0;
  //if(p->pagetable)
  if(p->thread_id != 0 && p->pagetable!=0){   //deallocating child's resources if thread_id!=0
    uvmunmap(p->pagetable, TRAPFRAME - PGSIZE *(p->thread_id), 1, 0);
  }
  else if(p->pagetable != 0)
  {
    proc_freepagetable(p->pagetable, p->sz);
  }
  p->pagetable = 0;
  p->sz = 0;
  p->pid = 0;
  p->thread_id = 0;
  p->parent = 0;
  p->name[0] = 0;
  p->chan = 0;
  p->killed = 0;
  p->xstate = 0;
  p->state = UNUSED;
}
```

- The core logic for thread creation is implemented in the make_clone() function, which takes a user stack pointer as input. Initially, the function performs a sanity check on the input stack parameter and returns -1 if the stack is NULL. Next, the function calls allocproc_thread() to allocate a process slot for the new thread. If it returns 0, indicating no available entries in the process table, the function exits early. Instead of creating a new address space, we copy the parent's address space to the child, enabling the thread to share memory with its parent. In the following lines, we copy the saved user registers (trapframe) and ensure the function returns 0 to the child thread. `np->trapframe->sp = (uint64)(stack + size);` This points the child's stack to the top of the passed stack region.In the file descriptor loop, we increment the reference counts of all open file descriptors to share them with the child thread. Finally, the function returns the thread_id to the parent and 0 to the child thread. We set the child's user stack pointer using:

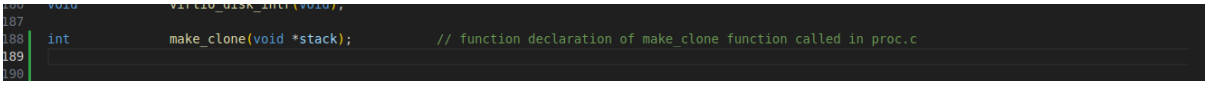
```c
int
make_clone(void *stack)
{
  int i, thread_id;
  struct proc *np;
  struct proc *p = myproc();
  int size = 4096*sizeof(void);
  if(stack == NULL)   // checking if stack is null or not
  {
    return -1;
  }

  if((np = allocproc_thread()) == 0)
  {
    return -1;
  }

  np->pagetable = p->pagetable;   //copying pagetable of parent

  //
  if (mappages(np->pagetable, TRAPFRAME - (PGSIZE * np->thread_id), PGSIZE, (uint64)(np->trapframe), PTE_R | PTE_W) < 0)
  {
    uvmunmap(np->pagetable, TRAMPOLINE, 1, 0);
    uvmfree(np->pagetable, 0);
    return 0;
  }

  np->sz = p->sz;

  *(np->trapframe) = *(p->trapframe);

  np->trapframe->sp = (uint64)(stack + size);

  np->trapframe->a0 = 0;

  for(i=0; i<NOFILE; i++)
  {
    if(p->ofile[i])
      np->ofile[i] = filedup(p->ofile[i]);
    np->cwd = idup(p->cwd);
  }
  safestrcpy(np->name, p->name, sizeof(p->name));

  thread_id = np->thread_id;

  release(&np->lock);  //releasing the lock

  acquire(&wait_lock); //acquiring the lock
  np->parent = p;
  release(&wait_lock);  //releasing the lock

  acquire(&np->lock);
  np->state = RUNNABLE;
  release(&np->lock);

  return thread_id;
}
```

- In the exit() function, we are ensuring the file closes only if thread_id is 0 , i.e , we do not close the file descriptors for the threads (thread_id != 0). Here we only call reparent() if its not a thread, i.e , if thread_id is 0.



```c
void
exit(int status)
{
  struct proc *p = myproc();

  if(p == initproc)
    panic("init exiting");

  // Close all open files.
  // for(int fd = 0; fd < NOFILE; fd++){
  //   if(p->ofile[fd]){
  //     struct file *f = p->ofile[fd];
  //     fileclose(f);
  //     p->ofile[fd] = 0;
  //   }
  // }

  //if thread_id is 0 then we close the files but for threads we don't close
  if(p->thread_id == 0){
    for(int fd = 0; fd<NOFILE; fd++)
    {
      if(p->ofile[fd])
      {
        struct file *f = p->ofile[fd];
        fileclose(f);
        p->ofile[fd] = 0;
      }
    }
  }

  begin_op();
  iput(p->cwd);
  end_op();
  p->cwd = 0;

  acquire(&wait_lock);

  // Give any children to init.
  //reparent(p);

  if(p->thread_id == 0)
  {
    reparent(p);
  }

  // Parent might be sleeping in wait().
```

- defs.h
  - Here, we are declaring make_clone() function which is called in sysproc.c.



```c
int             make_clone(void *stack);          // function declaration of make_clone function called in proc.c
```

- trap.c

- We explicitly specify the new TRAPFRAME location for child threads to the kernel. If the `thread_id` is `0`, the default TRAPFRAME address is used instead.

```
//telling kernel the new trapframe locations for child thread
//if  p->ID is 0 then default TRAPEFRAME address is used
((void (*)(uint64,uint64))trampoline_userret)(TRAPFRAME - (PGSIZE * p->thread_id), satp);
}
```

- usys.pl
  - Adding new system call, clone in the user level.

```
entry("sleep");
entry("uptime");
entry("clone");                    #entrying clone as a system call
```

- thread.h
  - In this file, we declare a `struct lock_t` along with two routines: `lock_acquire()` and `lock_release()`, which are used to acquire and release the lock, respectively. Additionally, we declare `thread_create()` and `lock_init()` functions to create a new thread and initialize the lock.

```
user > C thread.h > ...
#include "kernel/types.h"
typedef struct lock_t {                              //declaring struct lock_t
uint locked;
}lock_t;
int thread_create(void *(*start_routine)(void*), void *arg);    //
void lock_init(lock_t *lock);                        //declaring function to set the locked variable as 0 initially
void lock_acquire(lock_t *);                         //declaring fucntion to acquire the lock
void lock_release(lock_t *);                         //declaring function to release the lock
```

- thread.c
  - In this file, we implement a thread library routine for creating child threads.

    - **`thread_create()`**
      This function allocates a user stack and creates a child thread.
    - If `thread_id` is `0`, the child thread is successfully created and `start_routine()` is executed.
    - If `thread_id` is `-1`, thread creation fails and the function returns `0`.
    - **`lock_init()`**
      Initializes the lock by setting the `locked` variable to `0` for every thread created.
    - **`lock_acquire()`**
      Attempts to acquire the lock:
    - If `locked` is `0`, the thread acquires the lock and sets `locked` to `1`.
    - If `locked` is `1`, the thread “spins” (busy-waits) until `locked` becomes `0`.
    - **`lock_release()`**
      Releases the lock by setting the `locked` variable back to `0`.

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/spinlock.h"
#include "user/thread.h"
#include "user/user.h"

//wrapper function of clone to allocate a user stack and create a child thread
int thread_create(void *(*start_routine)(void*), void *arg)
{
    int thread_id;
    int stack_size = 4096*sizeof(void);          //size is PGSIZE
    void* stack = (void*)malloc(stack_size);     //creates user stack of PGSIZE bytes
    thread_id = clone(stack);                          //creates a new thread, thread_id is returned
    if(thread_id == 0)                                 //check if the reurned value is 0
    {
        (*start_routine)(arg);                   //if child thread is successfuly created then,
        exit(0);                                 //start_routine is called to continue execution of child
    }
    return 0;                                    //returns 0 if child creation is a failure
}

//init function to set the locked vairable as 0 for every thread
void lock_init(struct lock_t *lock)
{
    lock->locked = 0;       //set locked as 0 initially
}

//function to acquire the lock is locked is 0
//if it is 1 then it'll wait till it becomes 0
//then the lock is acquired and locked is changed to 1
void lock_acquire(struct lock_t *lock)
{
    while(__sync_lock_test_and_set(&lock->locked, 1) != 0);
    __sync_synchronize();
}

//function to release the lock
void lock_release(struct lock_t *lock)
{
    __sync_synchronize();
    __sync_lock_release(&lock->locked, 0); //locked is set to 0 once lock is released
}
```

- user.h
  - Declaring clone system call in the user level according to its return type and argument.

```c
int uptime(void);

int clone(void*);             //declaring system call clone
```

- Makefile

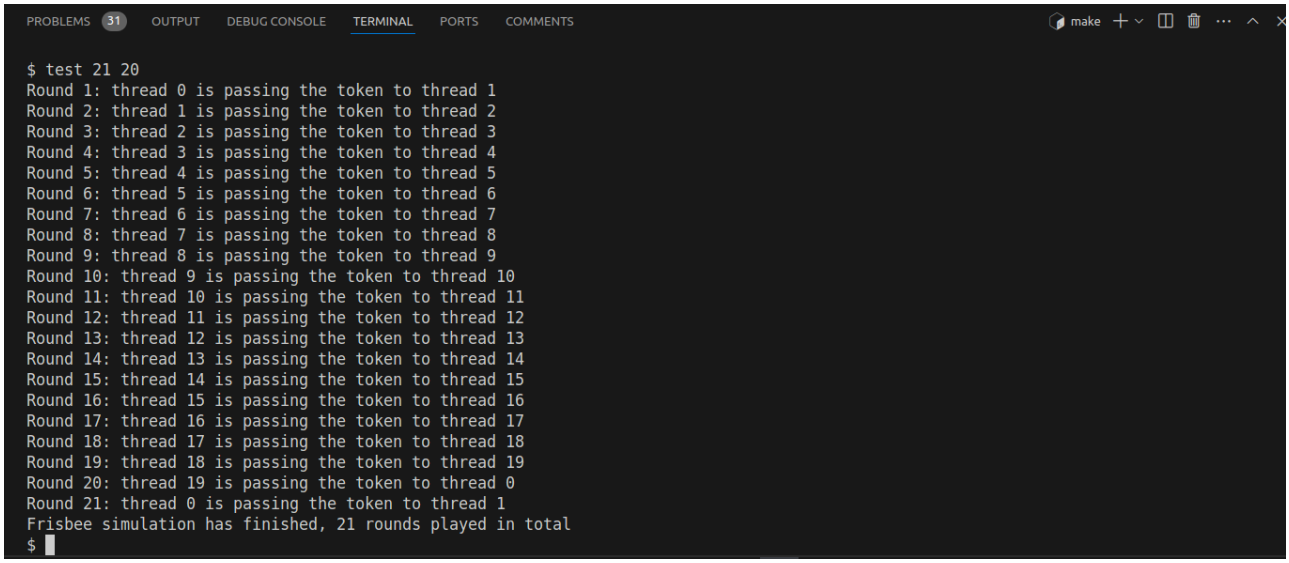
```
ULIB = $U/ulib.o $U/usys.o $U/printf.o $U/umalloc.o $U/thread.o

_%: %.o $(ULIB)
```

Results

- Output for 10 rounds with 3 threads:

```
PROBLEMS 31  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  COMMENTS                                make

rive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ test 10 3
Round 1: thread 0 is passing the token to thread 1
Round 2: thread 1 is passing the token to thread 2
Round 3: thread 2 is passing the token to thread 0
Round 4: thread 0 is passing the token to thread 1
Round 5: thread 1 is passing the token to thread 2
Round 6: thread 2 is passing the token to thread 0
Round 7: thread 0 is passing the token to thread 1
Round 8: thread 1 is passing the token to thread 2
Round 9: thread 2 is passing the token to thread 0
Round 10: thread 0 is passing the token to thread 1
Frisbee simulation has finished, 10 rounds played in total
$
```

- Output for 21 rounds with 20 threads:



Contributions

- Manoj: Implemented the logic to add the `clone()` system call for creating child threads.
- Vijay: Implemented a user-level thread library to create threads, and to acquire and release locks. Developed the necessary logic to support these functionalities.
- Subhash: Contributed to modifications in the `allocproc_thread()` and `exit()` functions within `proc.c`. Also assisted in updating the related documentation.